AAC ANDAMAN AERODROME

PLATFORM ARCHITECTURE OFFICE

Investor Documentation

The Andaman Aerodrome Network Platform — a turn-key platform for aviation activities built on a network of helipads and heliports, with white-label commercial services as its value-added layer.

DOCUMENT	ISSUED	VERSION	CLASSIFICATION
AAC-INV-001	June 2026	1.0	Confidential — Investor Use Only

AAC-INV-001 · Contents

Table of Contents

AAC-INV-001 · SECTION 01

Executive Summary

§ The asset

The company owns a production software platform — working title **Andaman Aerodrome Network Platform (AANP)** — that is the digital operating system for a network of helipads, heliports and aircraft. It provides, as a **turn-key solution for aviation activities**: the network graph (sites and routes), fleet and maintenance management, mission planning across activity types (tours, charters, transfers, medevac, ferry), crew assignment, disruption handling, and an automated scheduling and availability engine.

Layered on this base — and cleanly separated from it — is a **complete commercial suite**: product catalog, component-based dynamic pricing, event-sourced bookings, two integrated payment providers, customer identity, a reseller/commission network with Thai tax mechanics, marketing tooling and reporting. The commercial layer is built as **value-added services on the network platform**, and is white-label ready.

SilkSkyAir, the company's helicopter-tour business, is the **first commercial operator built on the platform** — and the proof that the entire stack runs a real company: acquisition through multilingual storefront and campaigns, real-time quoted bookings, card and PromptPay payments with automatic reconciliation, fulfilment through the network scheduling core, partner commissions settling automatically, and customers managed through their own portal.

§ Why it matters to this investment

The investment focus is the **expansion, ownership and management of the operational network** — heliports, helipads and aircraft. The platform converts that physical expansion into compounding digital value:

1. **Expansion is a data operation.** A new heliport is a node and its route edges; a new aircraft is a fleet row. Both are immediately productive across scheduling, availability and every commercial product — the platform guarantees near-zero digital marginal cost for physical growth, with utilization from day one.
2. **One network, unlimited operators.** The platform's tenancy model — organizations, database-enforced isolation, module-shaped privileges, per-organization commissions and settlement — already supports hosting **any number of white-label commercial operators** on the shared network core. SilkSkyAir is operator #1 and the template; each additional operator approaches pure configuration cost.
3. **The separation is real, not aspirational.** Network and commercial concerns are partitioned in the database schemas, the privilege namespace and the repository structure. The same boundaries that enable white-labelling also give the company **structural options**: licensing

the commercial suite, joint ventures per market, or formally dividing the platform into a network company and commercial entities.

4. **Activity-agnostic by design.** Tours are one of five live mission types. Transfers, charters and medevac run on the same engine the day they are commercially relevant — new aviation businesses launch on infrastructure that is already amortized.

§ Why the platform is a high-value asset

The claims are verifiable in the source repositories, not asserted:

- **Scale of build:** 503 ordered database migrations across 16 schemas (140+ active tables), 8 deployable applications/services, 3 published shared libraries, 20 event-driven automation workflows, 26 permission-gated back-office modules, 19 repositories.
- **Modern and modular:** current-generation stack (Next.js 16/React 19, Astro, PostgreSQL/Supabase, Strapi 5, n8n), fully serverless operations, audience-specific apps over a strongly-governed data core, event-driven integration.
- **Risk already retired:** live payment, CRM and CMS integrations; staging/production discipline with E2E test orchestration; database-enforced security (RLS); tamper-evident audit trails via event sourcing; multilingual (EN/TH live, RU/ZH provisioned) and multi-timezone by schema design; no proprietary lock-in at the core.
- **Production-hardened:** the migration history, staff manuals, release compilations and client review cycles in this repository document a system shaped by real customers, partners and payment flows — the part of platform value that cannot be shortcut.

§ The key person

The platform was conceived, architected and delivered under a **single platform architect**, and the artifact shows it: one coherent design philosophy across 19 repositories, delivered at a velocity that multi-vendor teams rarely match, with the institutional knowledge of every boundary and its rationale concentrated in one role. Continuity is engineered (reproducible environments, ordered migrations, documentation), so the platform is transferable — but its **strategic direction and extension velocity remain coupled to its architect**. The platform de-risks continuity; the architect de-risks evolution. For an expansion-focused investment, both assets matter, and they compound together.

§ The program

The roadmap concentrates investment where the thesis points: grow the network (nodes and fleet, sequenced by yield data the platform already produces), activate adjacent activities (transfers, charters, medevac) on the existing engine, and scale the white-label operator program in four stages — second internal brand, first external operator, productized onboarding, structural options. Revenue surfaces opened by the program include capacity and network fees, white-label

licensing, transaction-level participation, operator-facing services, and the company's own commercial operations, with SilkSkyAir as the living showcase.

The platform is the multiplier on the network investment: every helipad, heliport and aircraft the company adds becomes immediately schedulable, sellable and shareable across an unlimited number of commercial businesses — on infrastructure the company already owns.

Full documentation: Index · next chapter: The Network Platform

AAC-INV-001 · SECTION 02

The Network Platform

> **Working title:** Andaman Aerodrome Network Platform (**AANP**). The name designates the core, operator-neutral layer of the system; it is a working title and can be re-branded without any technical change.

§ What the platform fundamentally is

AANP is the **digital operating system for a network of helipads, heliports and aircraft**. It is not a booking website with a flight database attached — it is the inverse: a complete aviation network management core, on top of which commercial products (tours, charters, transfers) are merely one category of consumer.

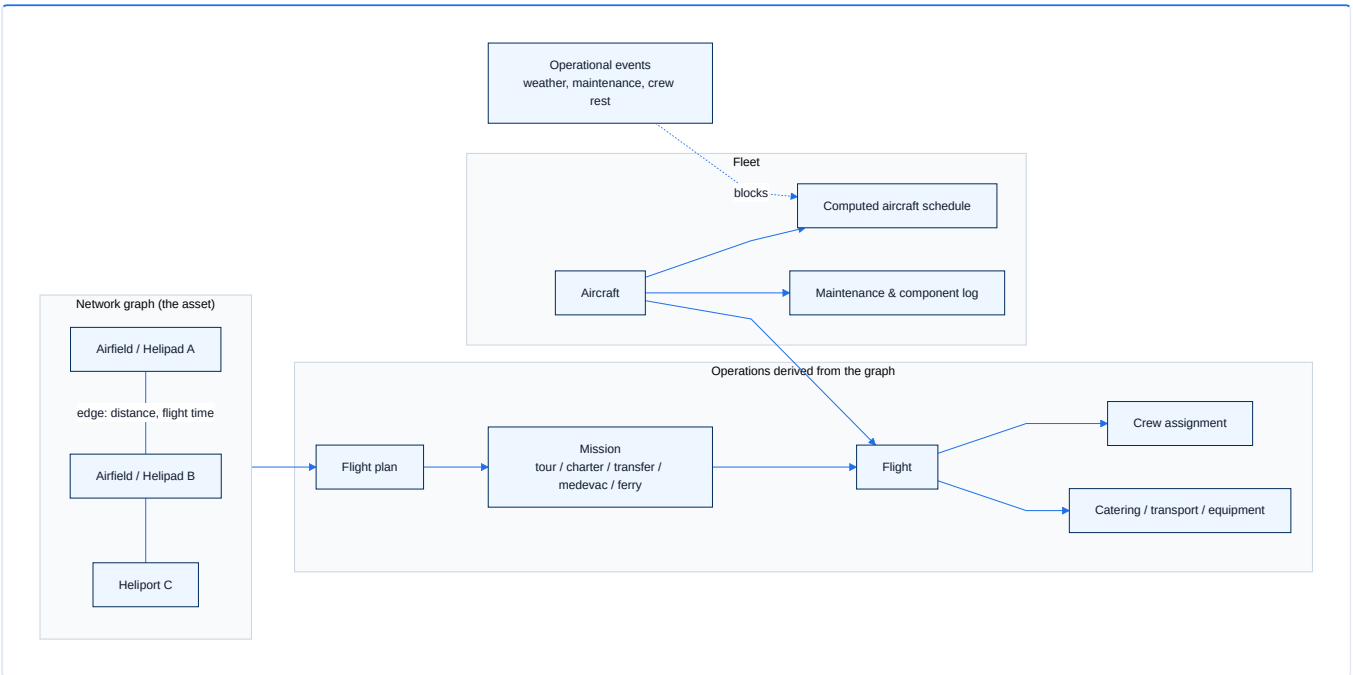
Everything an organization needs to **own, operate and expand a vertical-flight network** is a first-class citizen of the data model:

NETWORK CAPABILITY	PLATFORM IMPLEMENTATION
Helipads / heliports / airfields	`airfields` — location, timezone, contact, metadata
Route network between sites	`airfield_network_edges` — distance, flight time per leg
Aircraft fleet	`aircraft` — registration, model, capacity, default crew/components
Fleet maintenance & service log	`aircraft_events`, `aircraft_components`
Route planning	`flight_plans` — departure/arrival airfield, waypoints, duration
Work orders for any aviation activity	`missions` — typed: tour, charter, transfer, medevac, ferry
Scheduled operations	`flights` — aircraft, status lifecycle, departure time
Crew management	`flight_crew` — pilots, co-pilots, engineers, medics per flight
Flight logistics	`flight_components` — catering, ground transport, equipment
Network-wide disruptions	`operational_events` — maintenance, weather, crew rest, inspections

All of these live in the production database of `silkskyair-api` today (migrations `20251230300*` onward), protected by dedicated access privileges (`module:network:*`, `module:scheduling:*`) that separate network operations from every commercial concern.

§ The network is the graph; everything else follows

The model is deliberately graph-shaped: **airfields are nodes, network edges are routes**, and every flight the platform ever schedules is derived from that graph.



The consequence for expansion is structural: **adding a heliport to the network is a data operation, not a software project.** A new node plus its edges immediately participates in route planning, scheduling, availability and every commercial product downstream.

§ The scheduling and availability engine

The platform continuously answers the hardest operational question in vertical flight — *"which aircraft can be where, when, with whom on board?"* — automatically:

- `scheduler_rebuild` recomputes every aircraft's schedule (missions, ferry legs, returns, maintenance windows) for a time window. It runs nightly via `pg_cron` and is re-triggered immediately when bookings, network edges or maintenance events change.
- **Availability computation** enforces operational rules in real time: operational-event blackouts (per aircraft, per airfield, per crew member), booking cutoffs, join-flight cutoffs for shared flights, and seat-capacity checks — before any commercial logic runs.
- **A dedicated cache layer** (`cache` schema: month calendars, per-date slots) keeps availability queries fast at consumer-traffic volumes, with automatic invalidation when operational or pricing data changes.

This engine is operator-neutral. It schedules *missions* — a tour for a commercial brand, a medevac, a ferry flight and a private charter are the same kind of object to the core. That neutrality is what makes the platform a **turn-key solution for aviation activities** in general, rather than a tour-sales system.

§ Mission types: the full span of aviation activities

`mission_types` is an open taxonomy. The platform ships with:

TYPE	ACTIVITY
`tour`	Scheduled sightseeing products (the SilkSkyAir showcase — see case study)
`charter`	Private on-demand flights
`transfer`	Point-to-point passenger transport between network nodes
`medevac`	Medical evacuation operations
`ferry`	Repositioning flights, automatically planned by the scheduler

New activity types (cargo, survey, training, agricultural work) are additive rows, not new modules — the scheduling, crew, maintenance and disruption machinery applies unchanged.

§ Operational integrity by design

- **Event-sourced records** — operational state changes append events rather than overwrite rows, producing a complete audit trail (see Platform Asset Value for the compliance argument).
- **Role-segregated access** — network and scheduling functions are guarded by database-enforced privileges (`rls_has_privilege('module:scheduling:manage')` and related policies), so commercial users can never touch operational data.
- **Disruption handling as data** — an `operational_event` with a scope (aircraft, airfield or crew member) instantly and consistently blocks availability across every commercial channel, with no coordination required between teams.

§ Why this layer is the company's primary asset

1. **It encodes the physical network.** The helipads, heliports, routes and aircraft — the assets new management intends to own, expand and operate — have a one-to-one digital representation that is already running in production.
2. **It is activity-agnostic.** Tours are one of five mission types. The same core can operate transfers, medevac and charter businesses the day they are commercially relevant.
3. **It compounds.** Every node added to the graph increases the value of every other node (more routes, more products, more utilization options) — the classic network effect, captured in a schema built for it.
4. **It is separable.** The commercial features documented in the next chapter sit *on top of* this layer and can be white-labelled to any number of operators — see White-Label Separation.

AAC-INV-001 · SECTION 03

Commercial Services — Value Added on the Network

The commercial layer turns network capacity into revenue. It is a **complete, production-grade commerce stack** — catalog, dynamic pricing, bookings, payments, customers, resellers and marketing — built strictly *on top of* the network core described in The Network Platform. Nothing in this chapter is required for the network to operate; everything in this chapter monetizes it.

This separation is the basis of the platform's white-label strategy: the entire layer can be offered, as a package, to any commercial operator on the network (see White-Label Separation).

§ The commercial stack at a glance

CAPABILITY	PLATFORM IMPLEMENTATION
Product catalog	`tours` (+ `tours_i18n` translations, `tour_media`, departure points via `tour_airfields`)
Base & seat pricing	`tour_pricing` — per aircraft, seat capacity, shared-flight discounting
Real-time availability & quotes	availability engine + `cache` schema (operational rules first, pricing second)
Bookings	`bookings` + event-sourced lifecycle (`booking_events`, 40+ event types)
Line-item pricing	`booking_price_components` — base, share discount, promotion, coupon, private premium, surcharge, tax
Passenger manifests	`booking_passengers` — including per-passenger weight for flight safety
Add-on sales	`ancillary_items`, `booking_ancillary_items` (meals, transfers, insurance)
Payments	`payments` schema + `payment_intents` — Omise cards, SCB PromptPay QR, refund workflow
Customer identity	`member_profiles` (canonical, email-keyed), accounts, sessions, lifecycle events
Reseller network	`organizations` (type `partner`), commissions & settlement (`booking_commission_settlements`)
Marketing	`pricing.promotions`, `coupons`, `campaigns` (QR codes, landing pages), SkyStories editorial content
Reporting	`reporting` schema — SQL-template report engine with scheduled partner dispatches

§ Dynamic pricing, computed not configured

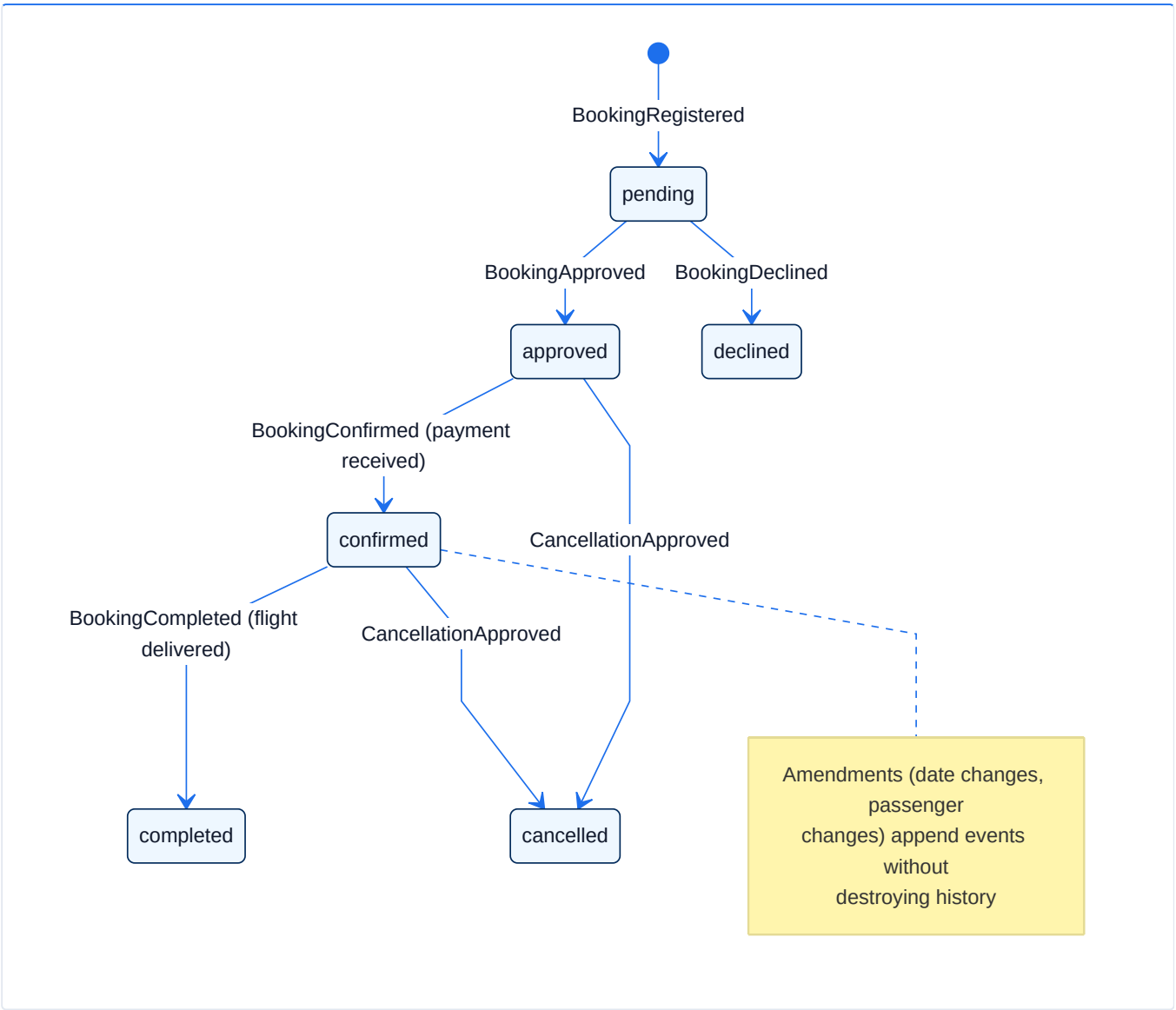
Every quoted price is assembled from explicit components at query time:

1. **Base** — `tour_pricing.base_amount` per seat for the assigned aircraft.
2. **Shared-flight discount** — automatic discount when a passenger joins an existing shareable flight, increasing load factor on already-committed capacity.
3. **Promotions** — time-boxed, scope-aware campaigns (`pricing.promotions`: percent, fixed or ancillary; constrained by booking window and travel window).
4. **Coupons** — redemption-coded discounts with usage limits and full audit (`coupon_redemptions`), optionally restricted to specific organizations.

Each component is persisted per booking as a `booking_price_components` line item — positive charges, negative discounts, with source references. The commercial ledger of any booking is therefore **self-explanatory and audit-ready**, and new pricing strategies (dynamic premiums, partner-tier pricing) are additive component types.

§ Event-sourced bookings: the lifecycle is the audit trail

Booking status is never stored as a mutable field. It is **derived from an append-only event log** (`booking_current_status()` over `booking_events`):



More than forty event types cover registration, approval, payment, amendment requests and approvals, cancellation and refund workflows, partner attribution and completion. The commercial benefit:

- **Disputes are resolvable from the record** — who changed what, when, is always known.
- **Compliance posture** — regulators and auditors get a tamper-evident history for free.
- **Workflow automation** — every event is a webhook trigger for the n8n workflow engine (confirmation emails, manager alerts, CRM sync, payment capture).

§ Payments and settlement

- **Multi-provider checkout** — Omise (cards) and SCB PromptPay (QR) are integrated end-to-end: `payment_intents` tracks tokens, sources and charges per attempt; provider webhooks reconcile status automatically. The platform stores no card data (tokenized, PCI-aligned).
- **Payment requests & refunds** — `payments.payment_requests` and `payments.booking_refund_requests` formalize money movement as workflow objects with status taxonomies, not ad-hoc operations.

- **Partner commissions** — bookings attributed to a reseller organization settle through `organization_bookings` and `booking_commission_settlements`: per-organization commission percentages, computed amounts, settlement status. The model already supports both settlement directions used in the field (operator-collected and partner-collected), with Thai VAT and withholding-tax mechanics implemented.

§ Customers as durable assets

`member_profiles` is a canonical, email-keyed customer identity that accumulates across channels: bookings, sessions, lifecycle events, marketing consent, acquisition source (first and latest). Members can affiliate with organizations (`member_organizations`) — the foundation for employee, affiliate and franchise relationships. The customer base is thus a **structured, portable asset**, not rows scattered across a booking table.

§ Marketing and content built in

- **Campaigns** — landing pages, short links and QR codes (`campaigns`, Short.io integration), published to the CMS automatically.
- **SkyStories** — an editorial subsystem (dedicated `skystories` schema: entities, media with video hosting, keywords, tour associations, engagement events) powering content-led acquisition.
- **Internationalization throughout** — a dedicated `i18n` schema plus companion translation tables (`tours_i18n`, `promotions_i18n`, `campaigns_i18n`, …). English and Thai are live; Russian and Chinese are provisioned in the locale model. New markets are content work, not engineering work.

§ Why "value added" is the right frame

Every capability above consumes the network core and adds margin on top of it:

```
network capacity (AANP)  →  products (tours/charters/transfers)  →  channels (web, members, partners)  →  payments & settlement  →  customer relationships  →  repeat revenue
```

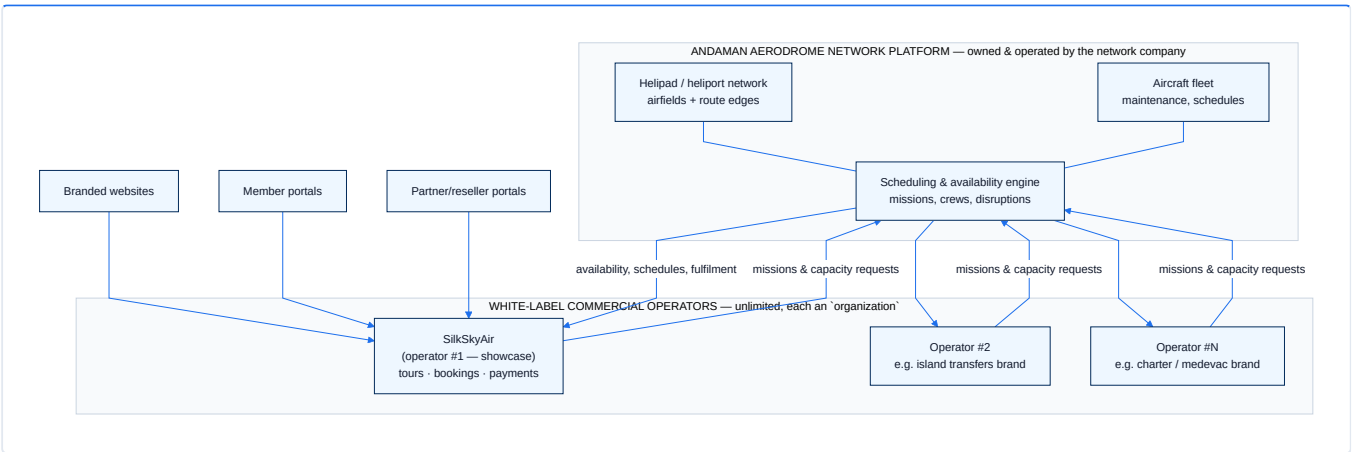
The layer is comprehensive enough to run a real business today — demonstrated by SilkSkyAir, the first operator — and modular enough to be replicated for the next operator without touching the network core.

AAC-INV-001 · SECTION 04

White-Label Separation — One Network, Unlimited Operators

This chapter shows **how the platform divides into a network platform plus an unlimited number of white-label commercial operators** — and why that division is a configuration of mechanisms that already exist in production, not a re-architecture.

§ The target operating model



The network company runs the base layer and sells **capacity plus a turn-key commercial suite**. Each commercial operator gets, under its own brand: a storefront, a booking engine with dynamic pricing, customer and reseller portals, payment rails, marketing tooling and reporting — while the network company retains the helipads, aircraft, scheduling and operational control.

§ The separation already exists in the schema

The division is not a roadmap item. The mechanisms are in the production database today:

1. ORGANIZATIONS ARE THE TENANT UNIT

- `organizations` is a first-class entity with a **type taxonomy** (`organization_types`: `partner`, `operator` — extensible by row insert), its own branding fields (name, slug, logo, contact), geographic scope (`organization_countries`), and a per-organization `commission_percent`.
- `organization_user_roles` and `organization_identity_roles` give every organization its own staff, roles and permissions.
- `organization_invitations` provides self-service team onboarding per tenant.

2. DATABASE-ENFORCED ISOLATION

- Row-Level Security policies gate every sensitive table through privilege checks (`rls_has_privilege('module:…:access' / ':manage')`, `rls_is_platform_admin()`) — 44 migrations build out this policy layer.
- The privilege namespace is **module-shaped** (`module:network:*`, `module:scheduling:*`, `module:bookings:*`, `module:partners:*`), which maps exactly onto the network/commercial boundary: network privileges stay with the network company; commercial privileges are granted per operator organization.

3. COMMERCIAL FLOWS ARE ALREADY ORGANIZATION-SCOPED

- `organization_bookings` ties bookings to an organization with its own commission and settlement status; `booking_commission_settlements` computes and audits the money split.
- `coupon_organizations` scopes marketing instruments to a tenant.
- `member_organizations` affiliates customers with organizations — with a database trigger enforcing one active partner affiliation per member.

4. THE APPLICATION LAYER IS MODULAR PER AUDIENCE

- The back office (`silkskyair-manager`) is built on a **module registry** — 26 registered modules (network, scheduling, bookings, pricing, promotions, partners, payments, reporting, …) whose visibility is controlled by the same `module:*` privileges. A network-company console and an operator console are *permission profiles over the same codebase*, not separate products.
- Customer-facing apps (`silkskyair-www` storefront, `silkskyair-member` portal, `silkskyair-partner` reseller portal) consume the shared backend through the same APIs and are themeable via the shared UI library (Tailwind theme) and CMS-driven content — the standard white-label recipe: same engine, per-brand skin and content space.
- The `i18n` schema is **context-aware** (per-application contexts with their own locale sets), so each branded deployment carries its own copy and languages.

---

§ What "onboarding operator #2" actually involves

Because the mechanisms exist, onboarding a new commercial operator is a provisioning checklist, not a development project:

STEP	MECHANISM	NATURE
1. Create the tenant	Insert `organizations` row (type, branding, commission terms)	Data
2. Provision staff	`organization_user_roles` + module privileges (commercial set only)	Data
3. Define the catalog	Tours/charters/transfers referencing network airfields & capacity	Data + content
4. Brand the channels	Deploy storefront/member/partner apps with operator theme + CMS space	Configuration
5. Connect money	Payment provider account per operator; settlement terms in org record	Configuration
6. Go live	Availability, scheduling and fulfilment served by the shared network core	—

The marginal cost of each additional operator approaches the cost of configuration and content — the defining economics of a platform business.

§ Degrees of separation, selectable per strategy

The same design supports three increasingly strong separation models, in order of effort:

1. **Logical separation (available now)** — all tenants in one deployment, isolated by organizations + RLS + module privileges. Right for onboarding speed.
2. **Branded-channel separation (available now)** — per-operator storefront/portal deployments (the apps are independently deployable today) over the shared core.
3. **Corporate separation (structural option)** — because network tables, commercial tables and applications are already partitioned by schema, privilege namespace and repository, the platform can be split into **two products owned by two entities** — the network platform and a commercial-suite licensee — along boundaries that already exist. This is the option that lets the company divide the platform cleanly if the network business and commercial businesses are ever held or sold separately.

§ SilkSkyAir: tenant #1 as proof

SilkSkyAir is not "the platform" — it is the **first commercial operator built on it**, exercising every white-label surface end-to-end in production: branded storefront, member portal, partner/reseller network with commission settlement, two payment providers, CRM integration and multilingual content. The full walk-through is in the case study.

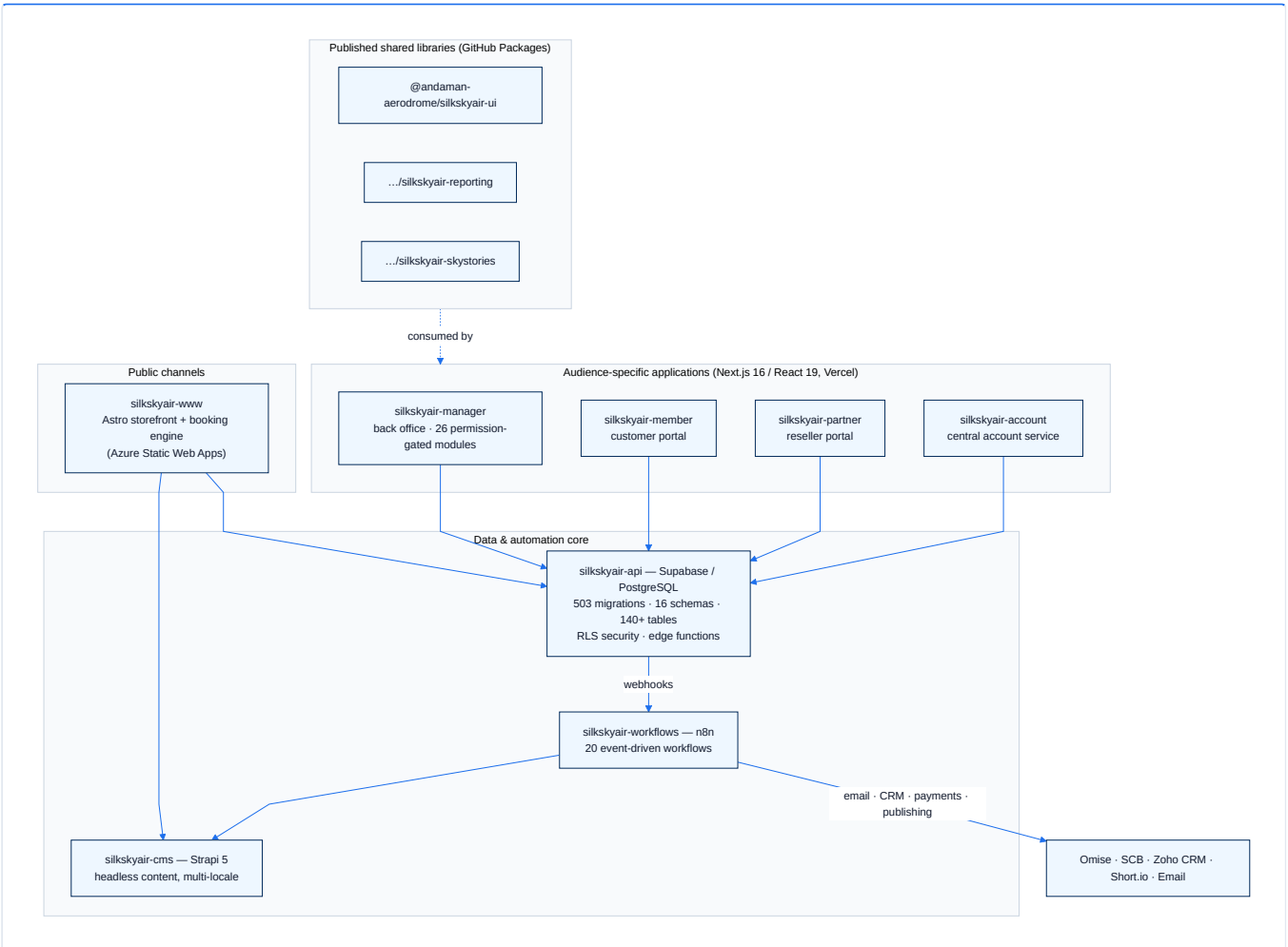
Its strategic role in this document is evidentiary: **everything an operator licensee would buy has already been built, integrated and operated for a real business.**

AAC-INV-001 · SECTION 05

Architecture — Modern and Modular by Design

The platform is built the way contemporary, venture-grade software is built in 2026: **small, independently deployable applications around a strongly-governed data core, with event-driven automation and serverless operations.** This chapter documents the technical substance behind the claims of modernity and modularity.

§ Platform topology



§ A modern stack, end to end

LAYER	TECHNOLOGY	WHY IT MATTERS TO AN INVESTOR
Frontends	Next.js 16 / React 19 / Tailwind CSS 4; Astro for the storefront	Current-generation frameworks; large hiring pool; long support runway
Data core	PostgreSQL on Supabase (managed)	Industry-standard database; no proprietary lock-in; portable by design
Security	Row-Level Security enforced **in the database**	Tenant isolation cannot be bypassed by application bugs
Automation	n8n workflow engine	Integrations are visual, auditable workflows — not buried code
Content	Strapi 5 headless CMS	Marketing iterates without engineering involvement
Hosting	Vercel, Azure Static Web Apps, Supabase Cloud, Strapi Cloud	Fully serverless: no servers owned, near-zero ops headcount, elastic scaling
Delivery	Git-based CI/CD, staging/production environments, E2E test orchestration (Playwright/Vitest)	Professional release discipline, verifiable in the repositories

§ Modularity, demonstrated at four levels

1. Repository level. The estate is 19 focused repositories — 8 deployable applications/services, 3 published libraries, plus orchestration, environment and documentation repos. Each piece versions, deploys and scales independently; none is a monolith that must be bought, sold or operated as an all-or-nothing block.

2. Schema level. The database is partitioned into 16 purpose-named schemas — network and operations (`operations`, `availability`, `workflows`, `cache`), commerce (`pricing`, `payments`, `partners`, `members`), growth (`analytics`, `reporting`, `skystories`), platform (`i18n`, `account`, `api`, `geo`) and `public` core. Domain boundaries are visible in the database itself, which is what makes the white-label separation credible.

3. Application-module level. The back office is a **module registry**: 26 modules (network, scheduling, bookings, pricing, promotions, partners, payments, campaigns, reporting, integrations, …), each gated by a `module:*` database privilege. Product packaging — which tenant sees which capability — is configuration.

4. Component level. Shared UI, reporting and content libraries are published as versioned packages and consumed by the apps like any third-party dependency. New applications (e.g. a future operator console for a licensee) start from a proven kit rather than from zero.

§ Event-driven by default

State changes in the core publish events; the workflow engine reacts:

- Booking registered → pricing components written, payment request raised, manager notified, CRM lead/deal created (Zoho).
- Payment webhook (Omise) → charge reconciled, booking confirmed, customer emailed.
- Campaign published → landing page pushed to CMS, short link + QR code generated.
- Nightly → aircraft schedules rebuilt (`pg_cron`), scheduled reports dispatched to partners.

The pattern means new integrations (a channel manager, an accounting system, a government reporting feed) attach to existing events — they do not require changes to the core.

§ Built for more than one market from day one

- Dedicated `i18n` schema with per-application contexts and per-context locale sets; translation companion tables across the catalog and marketing entities. English and Thai live in production; Russian and Chinese provisioned.
- Multi-currency-ready pricing fields; timezone-aware airfields and scheduling.
- Geography as data (`geo` schema, organization country scoping).

§ Engineering discipline as a due-diligence artifact

The repositories themselves are the evidence room:

- **503 incremental migrations** — every schema change reviewed, ordered and reproducible from zero.
- **Seed and environment tooling** — a complete local environment (`silkskyair-supabase`) with 30 dependency-ordered seed files; staging and production separation with documented promotion paths.
- **E2E orchestration** — a dedicated orchestrator repo running Playwright suites across all applications.
- **Operational documentation** — staff manuals, release compilations and weekly engineering reports in this repository.

A technical due-diligence team can verify every claim in this document directly against the codebase — which is precisely how this document was produced.

AAC-INV-001 · SECTION 06

Platform Asset Value

This chapter makes the asset case in terms a diligence team can verify: what exists, what it would take to recreate, what risks it retires, and where the value concentrates — including the key-person dimension.

No revenue figures or valuations are asserted here; those belong to management's financial materials. What this chapter provides is the **verifiable substance** any such figures would rest on.

§ What exists, measurably

All numbers below are derived directly from the repositories and can be re-counted at any time:

METRIC	VALUE	WHERE TO VERIFY
Database migrations (ordered, reproducible)	**503**	`silkskyair-api/supabase/migrations/`
Dedicated database schemas	**16**	schema creation migrations
Tables defined over the platform's life	**~160** (140+ in active service)	migrations
Seed files for full-environment reproduction	**30**	`silkskyair-api/supabase/seeds/`
Supabase edge functions	**3**	`silkskyair-api/supabase/functions/`
Event-driven n8n workflows	**20** across 7 categories	`silkskyair-workflows/workflows/`
Back-office modules (permission-gated)	**26**	`silkskyair-manager/lib/modules/registry.ts`
Repositories in the estate	**19**	the `andaman-aerodrome` GitHub organization
Published shared libraries	**3** (`ui`, `reporting`, `skystories`)	GitHub Packages
Deployable applications/services	**8** (manager, member, partner, account, www, cms, workflows, api)	repo estate
Languages	EN + TH live; RU + ZH provisioned	`i18n` schema and translation tables
Booking lifecycle event types	**40+**	`booking_event_types`

§ Replacement-cost reasoning

The honest way to price a software asset without invented numbers is to enumerate what a buyer would otherwise have to build. Recreating this platform requires, at minimum:

1. **An aviation network domain model** — airfields, route graph, fleet, maintenance, missions, crews, disruptions — plus a **scheduling and availability engine** with caching and nightly recomputation. This is the rarest part: it requires aviation domain knowledge, not just engineering hours.
2. **A complete commerce stack** — catalog, component-based dynamic pricing, event-sourced bookings, two payment-provider integrations with reconciliation, refunds, commissions with Thai VAT/withholding-tax mechanics, customer identity, promotions/coupons/campaigns.

3. **Five audience-specific applications** plus a headless CMS and a workflow engine, integrated end-to-end (email, CRM, payments, link/QR services).
4. **The operational hardening that only production exposure produces** — the 503 migrations are a fossil record of real-world edge cases already paid for: amendment workflows, settlement disputes, cache invalidation, locale gaps, partner review cycles (documented in this repository's `plans/` and `manuals/`).

Item 4 is the moat. A competitor can hire a team and copy a feature list; it cannot shortcut the iteration history with live customers, partners, regulators and payment providers.

---

§ Risk already retired

Investors price risk; this platform has already retired several classes of it:

- **Technical risk** — the system runs in production with staging/production environments, CI/CD, E2E test orchestration and documented releases.
- **Integration risk** — payments (Omise, SCB), CRM (Zoho), CMS (Strapi), automation (n8n) are live integrations, not roadmap promises.
- **Compliance posture** — event-sourced bookings and settlements give tamper-evident audit trails; access control is enforced in the database (RLS), not just in app code; no card data is stored (tokenized payments).
- **Market-expansion risk** — multilingual, multi-timezone and multi-currency-ready by schema design; adding a market is content and configuration work.
- **Key-platform-vendor risk** — the core is standard PostgreSQL and open-source-centric tooling (Strapi, n8n); managed services are conveniences, not lock-ins.

---

§ Strategic optionality

Beyond its operating value, the platform carries **option value** that conventional booking software does not:

1. **White-label licensing** — the separation model turns the commercial layer into a sellable product for other operators on the network.
2. **Clean divisibility** — network and commercial layers can be held, financed or sold as distinct assets along boundaries that already exist in the schema, privilege model and repository structure.
3. **Activity expansion** — charters, transfers, medevac and ferry operations are existing mission types of the same engine; new aviation businesses launch on infrastructure that is already amortized.
4. **Network effects** — each new helipad/heliport node increases the value of every existing node and of every operator on the platform.

§ The Platform Architect — key-person value

The platform was conceived, designed and delivered under a **single platform architect**, and that fact is visible in the artifact itself:

- **Coherence.** One design philosophy runs through 19 repositories: the same separation of network and commerce, the same module/privilege naming, the same event-driven patterns, the same documentation discipline. Systems assembled by rotating teams do not look like this; the absence of architectural drift is itself evidence of concentrated authorship.
- **Velocity.** The measurable output above — 503 migrations, 8 deployed applications/services, 3 published libraries, 20 production workflows, multilingual content systems — was produced at a pace and consistency that a hired multi-vendor team would struggle to match at any budget.
- **Institutional knowledge.** The architect holds the full map: why each boundary sits where it does, which constraints are aviation-domain requirements versus implementation choices, and how the white-label separation is meant to unfold. This knowledge is the difference between a platform that compounds and one that calcifies after handover.
- **Continuity plan, not bus-factor excuse.** The risk that key-person value usually implies is actively mitigated here: reproducible environments, ordered migrations, seed data, staff manuals and weekly engineering reports mean the platform is *transferable* — but its **strategic direction and extension velocity remain coupled to its architect**, which is precisely why retaining that role is part of the asset.

For an investor, the conclusion is straightforward: the platform and its architect are complementary assets. The codebase de-risks continuity; the architect de-risks evolution — the expansion into new nodes, new operators and new aviation activities that this investment thesis is about.

AAC-INV-001 · SECTION 07

Case Study — SilkSkyAir, the First Operator

SilkSkyAir is the platform's **reference implementation of a commercial operator**: a helicopter-tour business in the Andaman region, running end-to-end on the network platform described in the preceding chapters. Its role in this documentation is evidentiary — every white-label capability an operator licensee would buy is exercised here in production.

§ The business, as run on the platform

Acquisition → booking → payment → fulfilment → repeat, with no manual glue between the steps:

1. **Acquisition.** A multilingual storefront (`silkskyair-www` , Astro) serves the tour catalog, editorial SkyStories content, promotions and campaign landing pages with QR codes — all managed by the marketing team in the CMS, no engineering involved. Meta CAPI purchase attribution and CRM lead capture (Zoho) close the marketing loop.
2. **Booking.** The storefront's booking engine quotes real-time availability and a component-priced quote (base, shared-flight discount, promotion, coupon) computed by the platform's availability engine — operational constraints first, pricing second. Passenger manifests capture per-passenger weight, an aviation-safety requirement enforced at the data model.
3. **Payment.** Customers pay by card (Omise) or Thai PromptPay QR (SCB); webhooks reconcile charges and confirm bookings automatically. Refunds and additional payment requests are workflow objects with full audit trails.
4. **Fulfilment.** Confirmed bookings become missions on the network core: aircraft assigned, crews scheduled, catering and ground transport attached, schedules rebuilt nightly and on every relevant change. Operational disruptions (weather, maintenance) block availability across all channels instantly.
5. **Relationship.** Customers get a member portal (`silkskyair-member`) for itineraries, amendments and payments; their identity, history and consent accumulate in canonical member profiles.

§ The reseller network

SilkSkyAir distributes through hotels, agencies and other partners — managed on the platform's organization model:

- Partners are `organizations` with their own staff, roles and portal (`silkskyair-partner`).
- Every partner-attributed booking settles automatically through the commission engine: per-organization commission percentages, both settlement directions (operator-collected and partner-collected), Thai VAT and withholding-tax mechanics, and a full settlement audit trail.

- Scheduled reports are generated and dispatched to partners by the platform's reporting engine.

This is the same machinery that, generalized, becomes the **white-label operator program** described in White-Label Separation — partners are to SilkSkyAir what commercial operators are to the network platform, one level up.

§ Operated, not just launched

The depth of operational reality is visible in this documentation repository itself:

- Staff training manuals for partner, member, booking and content workflows (`manuals/domains/`), organized into weekly release compilations.
- Client review cycles with itemized remediation plans (`plans/`), weekly engineering reports, and production release inventories.
- Amendment workflows, cancellation flows, check-in handling, customer notes and payment edge cases — all shipped in response to real operational demand.

§ What this proves to an investor

CLAIM IN THIS DOCUMENTATION	SILKSKYAIR EVIDENCE
The network core can fulfil commercial demand	Tours scheduled, crewed and flown through the missions/flights engine
The commercial suite is complete	Catalog → dynamic pricing → booking → payment → fulfilment → repeat, live
The white-label surfaces work	Branded storefront, member portal, partner portal, CMS space, own payment accounts
Multi-party money flows settle correctly	Commission engine running with real partners, VAT/WHT mechanics included
The platform sustains operations	Release cadence, training manuals, review cycles documented in this repo

SilkSkyAir is therefore the strongest possible sales asset for the operator program: **the demo is a real company.** The next operator does not buy promises; it buys the system a working business already runs on — and the network company retains the infrastructure both stand on.

AAC-INV-001 · SECTION 08

Expansion Roadmap

This chapter translates the platform's design into the expansion program the investment is meant to fund: growing the **physical network** (heliports, helipads, aircraft), and multiplying the **businesses operating on it**. Mechanics only — commercial terms and financial projections belong to management's financial materials.

§ The expansion thesis in one line

> Every new node and every new aircraft added to the network is immediately productive across all operators and all activity types, because the platform turns physical expansion into a data operation.

§ Axis 1 — Network expansion (the primary focus)

Adding a heliport/helipad is, on the platform side: an `airfields` row, its route edges, and translated content. From that moment the node participates in route planning, scheduling, availability and every operator's catalog. The expensive part of expansion is land, regulation and construction — the platform guarantees the *digital* marginal cost of a node is near zero, and that utilization can begin on day one.

Adding an aircraft is an `aircraft` row with capacity, default crew and components; maintenance logging and nightly scheduling apply automatically. Fleet growth requires no software work.

Network density compounds. Each new node creates routes to every reachable existing node; transfers and multi-stop products that were impossible become sellable inventory. The graph model means the platform measures and exposes this directly (route edges, utilization, availability) — giving network management the data to sequence expansion by yield rather than intuition.

§ Axis 2 — Activity expansion

The mission-type taxonomy makes new aviation business lines additive:

ACTIVITY	STATUS ON THE PLATFORM
Tours	Live (SilkSkyAir)
Charters	Mission type live; charter back-office module exists
Transfers	Mission type live; becomes compelling as network density grows
Medevac	Mission type live; an operator/contract away from activation
Ferry / repositioning	Automated by the scheduler today
Future (cargo, survey, training, …)	Additive taxonomy rows on the same engine

§ Axis 3 — Operator expansion (the white-label program)

The separation model defines the product; the rollout is deliberately staged:

Stage 1 — Second internal brand. Launch one additional commercial brand (e.g. a transfers brand) as a separate `organization` on the shared deployment. Purpose: exercise the tenant boundary end-to-end and produce the operator-onboarding playbook from a live run, at zero external risk.

Stage 2 — First external operator. Onboard an independent operator under the playbook: own branding, catalog, payment accounts, staff and reseller network — on network capacity under commercial agreements. Purpose: validate the licensing motion and the support model.

Stage 3 — Operator program at scale. Productize onboarding (tenant provisioning, theming, CMS spaces, payment connection) into a repeatable package; the marginal operator approaches pure configuration cost. At this stage the company operates a genuine two-sided platform: network capacity on one side, commercial operators on the other.

Stage 4 — Structural options. With the program proven, the already-existing boundaries (schemas, privilege namespaces, repositories) support whichever structure the company chooses: licensing, joint ventures per market, or formal division into a network company and commercial entities.

§ Revenue surfaces this roadmap opens

Stated qualitatively — each is a mechanism the platform already supports:

- **Capacity & network fees** — operators consume scheduled capacity on network infrastructure (the missions/flights engine is the metering point).
- **White-label licensing** — the commercial suite as a product, per operator.
- **Transaction-level participation** — the commission/settlement engine generalizes from partner commissions to platform-level participation in operator bookings.

- **Value-added services** — payments, reporting, marketing tooling and CRM integration as operator-facing services on the shared core.
- **Direct commercial operations** — SilkSkyAir and future internal brands continue as operating businesses and as the program's living showcase.

§ Platform work that supports the roadmap

The honest engineering view: the foundations exist, and the roadmap concentrates remaining work where it belongs —

- **Operator provisioning productization** — turning the Stage-1 playbook into tooling (tenant setup, theming, CMS space creation) as the operator program scales.
- **Partner portal build-out** — the reseller portal is live at foundation level and deepens with each release cycle (its review-driven evolution is documented in this repository).
- **Locale completion** — Russian and Chinese content completion for market expansion (the schema and tooling already support them).
- **Continued network instrumentation** — utilization and yield analytics on the existing `analytics` / `reporting` schemas as the node count grows.

None of these are architectural risks; they are the scheduled deepening of systems whose hard parts — the network core, the tenancy model, the money flows — are already in production.